1. Ownership and valid state

Lesson 01 - 28 October 2026 Objective: explain how an object protects its rules when callers supply mutable data, retain references, and read the accepted data later. Java is the implementation language; ownership and resource budgeting transfer to other languages.

The original case

```
final class UploadRequest {
    private final String ownerId;
    private final List<String> allowedTypes;
    private final byte[] content;
    UploadRequest(String ownerId, List<String> allowedTypes,
                  byte[] content) {
        this.ownerId = ownerId;
        this.allowedTypes = allowedTypes;
        this.content = content;
    }
    List<String> allowedTypes() { return allowedTypes; }
    byte[] content() { return content; }
}
```

Requirements: an owner must be present; accepted content must remain unchanged during background processing; callers must not change the accepted MIME-type policy; documents may be 100 MB, so memory cost matters. The class does not enforce these requirements. It accepts an absent owner and retains caller-owned mutable objects. Its getters expose those same objects. A caller can modify the list or array either through its original reference or through a getter.

Why returning this.field does not fix it

In these getters, return allowedTypes; and return this.allowedTypes; return the same reference. The keyword this identifies the current instance. It does not copy, freeze or transfer ownership of an object. final fixes the field reference after construction. It does not freeze the object that the reference points to. Private fields hide direct field access, but exposing their mutable objects can still break encapsulation.

2. References, mutation and reassignment

A reference variable identifies an object. A list entry can hold a reference to that same object. Storing a reference does not automatically capture a snapshot of the object contents.

```
StringBuilder name = new StringBuilder("pdf");
List<StringBuilder> values = List.of(name);
name.append("-changed");
System.out.println(name); // pdf-changed
System.out.println(values.get(0)); // pdf-changed
```

Line 1 creates one mutable StringBuilder. Line 2 stores a reference to that same builder in the list. Line 3 modifies the existing builder. There is still one builder, observed through two references. The list entry was not replaced; the object it refers to changed.

Reassignment behaves differently

```
StringBuilder name = new StringBuilder("pdf");
List<StringBuilder> values = List.of(name);
name = new StringBuilder("jpg");
System.out.println(name); // jpg
System.out.println(values.get(0)); // pdf
```

Reassignment points name at another object. The list keeps its independently stored reference to the original builder. Variables are not permanently bound together.

Java passes arguments by value

This is shared references, not pass-by-reference. Java always passes arguments by value. For an object argument, that value is a reference. A method receives a copy of the reference, so it can mutate the shared object but cannot reassign the caller variable.

```
static void change(StringBuilder value) {
    value.append("!");
    value = new StringBuilder("other");
}
StringBuilder original = new StringBuilder("pdf");
change(original);
System.out.println(original); // pdf!
```

Decision rule: distinguish changing an object from changing which object a variable refers to. Mutation of a shared object is visible through every reference to that object; reassignment of one variable is not.

3. String versus StringBuilder

Why the String example leaves the list unchanged

```
String name = "pdf";
List<String> values = List.of(name);
name = name + "-changed";
System.out.println(name); // pdf-changed
System.out.println(values.get(0)); // pdf
```

String is immutable. The expression produces a different string value and the assignment changes name to refer to it. It does not modify the original string. The list continues to refer to "pdf". StringBuilder is mutable: append changes its existing contents. That difference explains why the earlier builder example showed a changed value through the list.

Does name.toString() solve the problem?

```
StringBuilder name = new StringBuilder("pdf");
List<String> values = new ArrayList<>();
values.add(name.toString());
name.append("-changed");
System.out.println(values); // [pdf]
values.add("exe"); // allowed
values.set(0, "jpg"); // allowed
values.clear(); // allowed
```

For StringBuilder, toString returns a String snapshot. Later builder mutations do not change that string. The ArrayList itself remains mutable, however. Protecting element contents and protecting list structure are separate tasks.

```
List<String> values = List.of(name.toString());
```

This combines an immutable String snapshot with an unmodifiable list. The snapshot captures the builder contents at the moment toString executes; if the builder already contains "pdf-changed", that is what is captured. If name already is a String, toString adds no protection: String is already immutable. It also does not freeze an enclosing collection.

What the constructor actually receives

The UploadRequest constructor receives an existing List<String>. Its elements are already immutable strings. The remaining job is to protect which strings are in the accepted list and their ordering. List.copyOf addresses that boundary.

4. Unmodifiable lists and immutable data

List.of(elements) assembles an unmodifiable list from supplied elements. List.copyOf(collection) produces an unmodifiable snapshot of an existing collection. Both reject null elements and reject structural updates such as add, remove and set.

```
List<String> original = new ArrayList<>(List.of("pdf"));
List<String> policy = List.copyOf(original);
original.add("exe");
System.out.println(policy); // [pdf]
policy.add("jpg"); // UnsupportedOperationException
```

Neither operation promises a deep copy of its element objects. List.copyOf may reuse a suitable unmodifiable list internally. Rely on its contract, not on a guaranteed fresh allocation.

An unmodifiable view is different from a snapshot

```
List<String> original = new ArrayList<>();
original.add("pdf");
List<String> view = Collections.unmodifiableList(original);
List<String> snapshot = List.copyOf(original);
original.add("png");
System.out.println(view); // [pdf, png]
System.out.println(snapshot); // [pdf]
```

The wrapper blocks mutation through view, but reads still reflect the backing list. A caller retaining original can change what view exposes. The snapshot is insulated from subsequent structural changes to original.

Shallow protection

```
StringBuilder name = new StringBuilder("pdf");
List<StringBuilder> values = List.of(name);
name.append("-changed");
System.out.println(values.get(0)); // pdf-changed
```

The list cannot replace its entries, but its entry refers to a mutable builder. An unmodifiable container is not automatically a deeply immutable object graph. List<String> is effectively immutable when its structure is unmodifiable because strings themselves cannot change. Vocabulary: encapsulation protects state and rules through controlled access; defensive copying isolates mutable data at a boundary; an unmodifiable view blocks mutations through that view; a snapshot captures state at a point in time; deep immutability requires reachable state to be protected too.

5. Mutable elements and both boundaries

A copied list can still share permissions

```
final class Permission {
    private boolean allowed;
    Permission(boolean allowed) { this.allowed = allowed; }
    void revoke() { allowed = false; }
    boolean isAllowed() { return allowed; }
}
Permission permission = new Permission(true);
List<Permission> accepted = List.copyOf(List.of(permission));
permission.revoke();
System.out.println(accepted.get(0).isAllowed()); // false
```

The collection snapshot stores the same Permission object. To capture the boolean value independently, use an immutable value representation:

```
record PermissionSnapshot(boolean allowed) {}
List<PermissionSnapshot> accepted = List.of(
    new PermissionSnapshot(permission.isAllowed())
);
```

This record has only a primitive boolean and is immutable. Records in general are only shallowly immutable: a record holding a mutable list still needs an ownership policy for that list. Business decision: accepting a permission snapshot and checking the current permission during execution have different semantics. A stable snapshot does not establish that snapshot-based authorization is appropriate. Revocation requirements determine the rule.

Protecting only the getter is insufficient

```
final class Upload {
    private final byte[] content;
    Upload(byte[] content) { this.content = content; }
    byte[] content() { return content.clone(); }
}
byte[] input = {10, 20};
Upload upload = new Upload(input);
input[0] = 99;
System.out.println(upload.content()[0]); // 99
```

The getter returns a copy, but the constructor retained the caller array. Fixing an array-returning design requires a constructor copy and a getter copy. Conversely, copying only during construction still leaves a getter that returns the internal array as a mutation route. Ownership review: trace every route to a mutable object, including original arguments, returned objects and nested mutable elements. Do not review getters in isolation.

6. A protected upload request

The following version validates the owner, snapshots the small list, privately copies the bytes once, and exposes a read-only view rather than a new full array on every read. Examples use modern Java; List.copyOf requires Java 10 or later, isBlank requires Java 11, and records used earlier require Java 16 or later.

```
import java.nio.ByteBuffer;
import java.util.List;
import java.util.Objects;
final class UploadRequest {
    private final String ownerId;
    private final List<String> allowedTypes;
    private final byte[] content;
    UploadRequest(String ownerId,
                  List<String> allowedTypes,
                  byte[] content) {
        if (ownerId == null || ownerId.isBlank()) {
            throw new IllegalArgumentException(
                "ownerId is required");
        }
        this.ownerId = ownerId;
        this.allowedTypes = List.copyOf(allowedTypes);
        this.content = Objects.requireNonNull(
            content, "content").clone();
    }
    String ownerId() { return ownerId; }
    List<String> allowedTypes() { return allowedTypes; }
    ByteBuffer content() {
        return ByteBuffer.wrap(content).asReadOnlyBuffer();
    }
}
```

An absent or blank owner is rejected before a usable instance exists. The constructor also rejects a null collection, null collection elements and a null payload through the called APIs. Empty lists and empty payloads are not rejected here; those require explicit business requirements. List.copyOf prevents changes to the source list from changing the accepted policy. Cloning the array prevents subsequent caller mutations from changing accepted bytes. The internal array must remain private and must never be modified by this class after construction. The example enforces ownership and owner presence. It does not yet validate MIME syntax, verify the actual content type, impose a byte limit, persist acceptance or establish authorization. Those are separate contracts rather than implied guarantees.

7. Read-only access, memory and concurrency

A view without another payload copy

```
ByteBuffer bytes = request.content();
byte first = bytes.get(); // reads and advances this view position
bytes.put((byte) 42); // ReadOnlyBufferException
```

The read-only ByteBuffer shares the private backing bytes. It blocks writes through its API and does not expose a writable backing array. Each content() call returns a separate buffer with its own position and limit, so readers using separate returned buffers do not share cursor state. Read-only bytes do not make the buffer object entirely immutable: position and limit can change. Sharing the same buffer between threads still requires coordination. Separately obtained views avoid sharing that cursor state. If the underlying array were changed internally, read-only views would observe those changes; the ownership rule is essential.

The 100 MB trade-off

A 100 MB input and its constructor clone coexist, requiring roughly 200 MB for payload bytes alone at that moment, plus object and application overhead. Each additional cloned array adds about another payload size. Read-only views allocate small wrapper objects, not another 100 MB payload. Garbage collection may eventually reclaim the caller array after it is no longer reachable, but the request cannot force the caller to release it. Many concurrent large uploads can therefore exhaust heap even with individually correct encapsulation. A larger-scale ingestion design can stream content into controlled storage and retain metadata plus a storage reference. This introduces decisions about persistence, cleanup, access control, integrity, failure handling and when responsibility for the upload is accepted. A storage reference alone does not guarantee immutable content.

Stable state during background work

```
List<String> policy = new ArrayList<>(List.of("pdf"));
UploadRequest request = new UploadRequest("owner-01", policy, input);
policy.clear(); // does not change request.allowedTypes()
```

Retaining policy directly would allow the worker to observe the cleared list, depending on timing. Snapshotting removes that shared mutation path. Locks would require every participant, including the caller, to use the same coordination protocol. Copying is not an atomic snapshot of an arbitrarily changing input. The producer must avoid modifying the source list or array while the constructor copies it. Transfer the completed request through an appropriate concurrency mechanism, such as an executor or concurrent queue; immutability does not replace all publication and lifecycle rules.

8. Common questions and clarifications

1. Would returning this.element fix the getters?

No. In the shown getters it returns the same field reference. Explicit field qualification changes readability, not ownership or mutability.

2. Is this encapsulation? Is copyOf similar to of?

Yes. Encapsulation is the objective; defensive copying helps enforce it. Both list factories return unmodifiable lists, but of accepts elements while copyOf accepts a collection. Neither makes mutable element objects immutable.

3. Was the builder bound to the list? Is it pass-by-reference?

The variable and the list entry each hold a reference to the same builder. append mutates that builder. Java passed the reference value by value; it did not bind the variable itself to a list entry.

4. Would modifying a String affect the list too?

No String contents can be modified. An operation such as name = name + "-changed" reassigns name to a different string. The list retains its original reference. The builder example differs because append mutates an existing object.

5. Could toString replace copyOf?

For a builder, toString snapshots element contents into an immutable String. It does not protect a mutable list structure. List.of(builder.toString()) protects both levels for that one element. The existing List<String> input already has immutable elements, so copyOf protects the structural boundary.

6. What makes the implementation clean?

Each operation exists for a requirement: constructor validation establishes valid owner state; copyOf isolates policy structure; clone establishes private byte ownership; read-only access avoids a payload-sized copy per read. At larger scale, the ownership boundary can move from heap data to controlled storage.

Accuracy boundaries

An unmodifiable list and a deeply immutable object graph are different guarantees. Read-only buffer access and an immutable buffer object are different guarantees. A snapshot after construction and a consistent copy during concurrent modification are different guarantees. Preserve these distinctions when adapting the design.

9. Practice and verification

These are follow-up exercises, not claims that they have been completed. Predict results before running code. Standard collection examples need java.util imports.

Exercise A - separate the two mutations

```
StringBuilder item = new StringBuilder("pdf");
List<StringBuilder> list = List.copyOf(List.of(item));
item.append("!");
item = new StringBuilder("png");
System.out.println(list.get(0));
```

Explain which object the list still refers to and which operation changed that object.

Exercise B - protect the whole boundary

A constructor copies a List<StringBuilder> using copyOf and returns that list directly. Describe what remains mutable. Redesign it so the accepted text is stable. Explain what information your new representation preserves or loses.

Exercise C - evaluate a large payload

Twenty caller arrays of 100 MB and twenty private clones remain live simultaneously. Estimate the payload-only heap requirement. Explain why a read-only getter does not remove that constructor-time cost. Identify the ingestion and acceptance rules needed before moving payloads to storage.

Exercise D - evidence for the upload contract

Verify four behaviours: modifying the original policy after construction leaves the request unchanged; modifying the original byte array leaves accepted bytes unchanged; mutating the returned policy is rejected; writing through the returned buffer is rejected. Also verify that two separately returned buffers have independent positions.

Verification and references

The protected UploadRequest and its ownership checks were compiled and run on Java 21. Other illustrative snippets and exercises are untested as printed; outputs shown are expected results. No heap benchmark, concurrency stress test, or storage implementation was performed.

Java 21 List API

Java 21 ByteBuffer API

Written by Alby Raju with AI assistance. Examples are educational and independent of employer systems.